

Phase 2 Roadmap: Kafka & WebSockets

Turning your JWT auth + job-board API into an event-driven, real-time backend — as a guided learning project.

Prepared for Yasowant | September 18, 2026

This is a planning document, not code. It looks at what your **node-auth-job-be** project already does, explains Kafka and WebSocket concepts in plain terms, and lays out a Phase 2 build plan that adds them to real features of this project — so the concepts stick because you're solving an actual problem, not a toy example.

1. Where the project stands today

Repo: **node-auth-job-be** — a JWT authentication + job-board REST API on Express 5 and MongoDB (Mongoose 9). Access/refresh tokens live in `httpOnly` cookies, passwords are hashed with `bcrypt`, CSRF protection and rate limiting are already wired in, and the project ships with Docker, Jest/Supertest tests and CI/CD. This is a genuinely solid REST foundation.

Domain models

Model	Purpose	Notable fields
User	Account + candidate profile	role (USER / RECRUITER / ADMIN), skills, experience, resume, tokenVersion, resetPasswordToken
Company	Recruiter's organization	owner, verified / verifiedBy, locations, industry
Job	Job posting	status (DRAFT→ACTIVE→CLOSED...), applicantCount, viewCount, text-indexed search

What each module currently does

Module	Endpoints	Notes
Auth (/api/auth)	register, login, refresh, forgot/reset-password, change-password, logout, logout-all, me, users (admin), profile	Full credential lifecycle; login and password-reset are rate-limited
Company (/api/company)	create, list, get, update, verify (admin), delete	Recruiter-owned, admin verification step
Job (/api/jobs)	create, list, get, my/jobs, update, delete	Recruiter-only writes, public/optional-auth reads, text search index

What's notably missing — and why that's good news

There is currently **no Job Application model** (candidates can't actually apply to a job yet — `applicantCount` exists on Job but nothing increments it), **no notifications**, and **nothing asynchronous or real-time** anywhere in the codebase. Every request is handled synchronously, start to finish, inside one

Express route handler.

That gap is exactly the opening Phase 2 needs: Kafka and WebSockets only make sense once there's a real event worth reacting to. Building the Applications feature first gives you that event.

2. The concepts, in plain terms

Message queue / Apache Kafka

Right now, when a request comes in, one function does everything before responding: validate → write to Mongo → maybe send an email → respond. If the email provider is slow, the whole request is slow. A **message queue** breaks that chain: the request handler just drops a small message ("user registered", "application created") onto a queue and returns immediately. Separate processes, called **consumers**, pick messages off the queue whenever they're ready and do the slow work.

- **Producer** — the code that publishes a message (usually inside your existing controllers).
- **Topic** — a named stream of related messages, e.g. application-events.
- **Partition** — a topic is split into ordered partitions so Kafka can parallelize across consumers while keeping order within a partition (e.g. per userId).
- **Consumer / consumer group** — a process that subscribes to a topic. Multiple consumers in the same group share the work; consumers in different groups each get every message independently (this is how one event can trigger email + a live socket push + an audit log, all separately).
- **Offset** — Kafka's bookmark of how far a consumer has read; this is what lets a crashed consumer resume without losing or (usually) re-reading everything.

Kafka specifically (vs. a simpler queue like RabbitMQ or Redis lists) is built for **high-throughput event streams that multiple independent consumers replay** — which is a good conceptual fit for "lots of things happen on a job board, and several different systems care about each one."

WebSockets / Socket.IO

HTTP is request■response: the client asks, the server answers, the connection ends. There's no way for your Express server to say "hey, someone just applied to your job" unless the browser keeps asking (polling). A **WebSocket** is a single long-lived, two-way connection — once open, the server can push a message to a specific browser at any time, with no new request. **Socket.IO** is a library that adds reconnection, rooms, and fallbacks on top of raw WebSockets, and is the standard choice for Node/Express.

How the two fit together

In this project, Kafka is the backbone that carries "something happened" between backend services; Socket.IO is the last mile that gets the relevant slice of that news to the one open browser tab that cares. The typical path for one event:

```
Express route (producer)
  → publish event to Kafka topic          e.g. application.created
  → HTTP response returns immediately    (fast, unblocked)

Kafka topic: application-events
  → notification-worker (consumer) → save Notification doc + send email
  → notification-worker             → emit via Socket.IO to room user:<id>
  → audit-worker (consumer)         → append to AuditLog collection
```

Same event, three independent reactions, none of them blocking the original request — that's the whole point of decoupling with a queue.

3. Phase 2 feature plan

Each row is a feature you can build and actually use, with Kafka and/or WebSockets doing real work in it — not bolted on for the sake of it.

#	Feature to build	Kafka's job	Socket.IO's job	What it teaches
1	Job Applications — Application model, apply endpoint, "my applications", recruiter applicant list	—	—	Foundation only. No event system is worth building until there's a real event ("someone applied").
2	Event-driven onboarding & notifications — publish <code>user.registered</code> , <code>application.created</code> , <code>application.status_changed</code> , <code>job.published</code>	Producer in each controller; a notification-worker consumer creates a Notification doc and sends the email	—	Producer/consumer basics, topic design, decoupling slow work (email) from the request cycle
3	Real-time notification delivery	Same consumer as above	The moment the consumer finishes, it emits to the user's private room over Socket.IO — bell icon updates with no refresh	JWT-authenticated socket connections, per-user rooms, bridging a queue event to a live push
4	Recruiter live dashboard — applicant count and "New applicant!" ticking up in real time on the job's page	Same <code>application.created</code> event, consumed again	Emits to a per-job / per-company room instead of a per-user one	Fan-out: one event, many differently-scoped listeners
5	Async view-count & search-relevance aggregation — stop writing to Mongo on every single job view	<code>job.viewed</code> events streamed in; a batching consumer flushes aggregated counts every few seconds	—	Why you batch/aggregate high-frequency events instead of one DB write per request; at-least-once delivery
6	Audit / activity log — immutable history of every write action	One consumer subscribed to every topic, appends to an AuditLog collection	—	Multiple independent consumer groups reading the same topics; event-sourcing basics
7	Presence + recruiter↔candidate chat (<i>stretch</i>)	Optional: chat messages can also be produced onto a topic for durability/audit	Namespaces, rooms, typing indicators, online/offline presence	Scaling sockets beyond a single process (Redis adapter)

4. Suggested build order

Six milestones, each one runnable and demo-able on its own before you move to the next — resist the urge to wire Kafka in on day one.

- 1 **Applications module, plain REST.** Application model (job, applicant, status, coverNote), apply / withdraw / list endpoints, recruiter view of applicants per job. No queue yet — get the feature itself right first.
- 2 **Kafka running locally.** Add Kafka to docker - compose (KRaft mode needs no Zookeeper) plus `kafka-ui` so you can literally watch messages land on a topic in a browser — genuinely the fastest way to build intuition for what a topic/partition/offset actually look like.
- 3 **First producer.** Wire `kafka.js` into 2–3 existing actions (register, apply, job published). Start by just `console.log`-ing in a bare consumer — prove the message round-trips before building anything real on top.
- 4 **Real consumers as separate processes.** `workers/notification-worker.js` run via its own npm script (`node workers/notification-worker.js`), writing a Notification model and sending real email (nodemailer or a provider's SDK).
- 5 **Socket.IO server.** Attach to the existing HTTP server, authenticate the socket handshake with the same JWT you already issue, join the user's private room on connect, and have the notification-worker push into it. Then add the recruiter's per-job room and the batching view-count consumer.
- 6 **Stretch:** audit-log consumer, presence tracking, recruiter↔candidate chat.

5. New pieces this adds to the project

Nothing here is written yet — this is the shape the finished project settles into.

```
docker-compose.yml      + kafka, kafka-ui  (add alongside existing services)

src/
  events/
    producer.js          thin wrapper around the kafkajs producer
    topics.js            topic-name constants, one place to see them all
  sockets/
    index.js             Socket.IO server, JWT auth, room join logic
  models/
    Application.js       NEW — job, applicant, status, coverNote
    Notification.js      NEW — recipient, type, payload, read
  controllers/
    applicationController.js  NEW
    notificationController.js NEW — list / mark-read

workers/
  notification-worker.js  separate long-running processes, not Express routes
  analytics-worker.js     view-count batching
  audit-worker.js         (stretch)
```

6. Quick glossary

Term	Meaning
Producer	Code that publishes a message to a Kafka topic (lives inside your Express controllers)
Consumer	A separate process that subscribes to a topic and reacts to each message
Consumer group	A named set of consumers sharing the read load of a topic; different groups each get every message independently
Topic / Partition	A named event stream, split into ordered partitions for parallelism
Offset	A consumer's bookmark of how far it has read in a partition
At-least-once delivery	Kafka's default guarantee — a consumer may see a message more than once, so handlers should be safe to run twice (idempotent)
WebSocket	A single long-lived, two-way TCP connection between browser and server
Socket.IO room	A label a socket can join so the server can broadcast to a subset of connected clients (e.g. one user, one job)

7. Reference docs

- Kafka fundamentals — kafka.apache.org/documentation
- kafkajs (Node.js Kafka client) — kafka.js.org
- Socket.IO — socket.io/docs

node-auth-job-be · Phase 2 planning document · generated with Claude